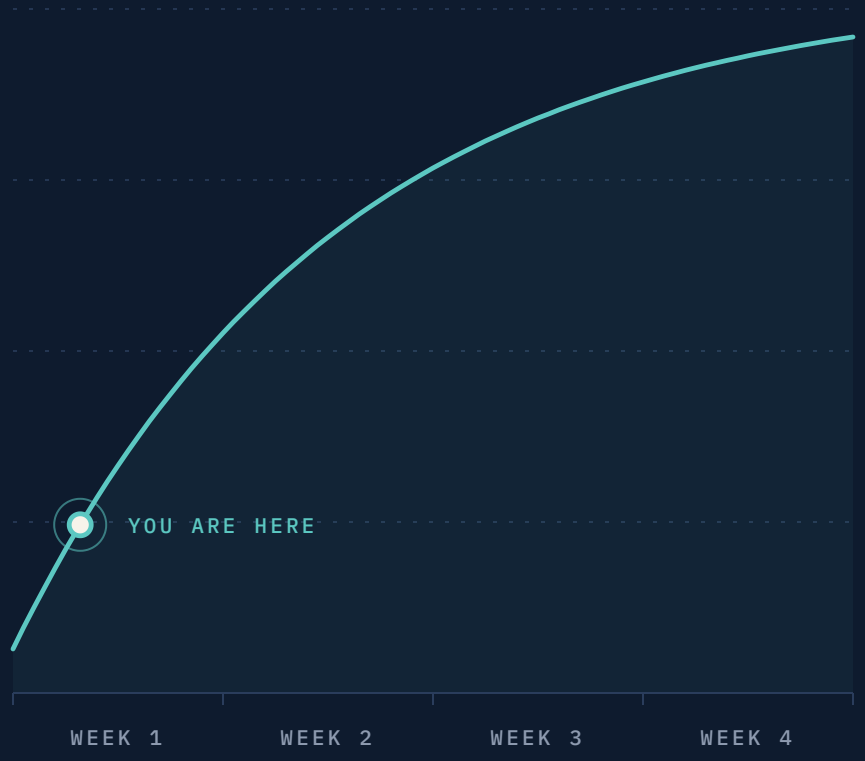
WEEK 1 · DAY 1, PART 2 · OVERVIEW

Python for Fixed Income

Introduction, Part 2



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Third Note

The book is in Python. Now make it work: tell me which of our four bonds are investment grade, cost every position in one go, and turn the coupon sum into something I can reuse.

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

Your Route Through Day 1, Part 2

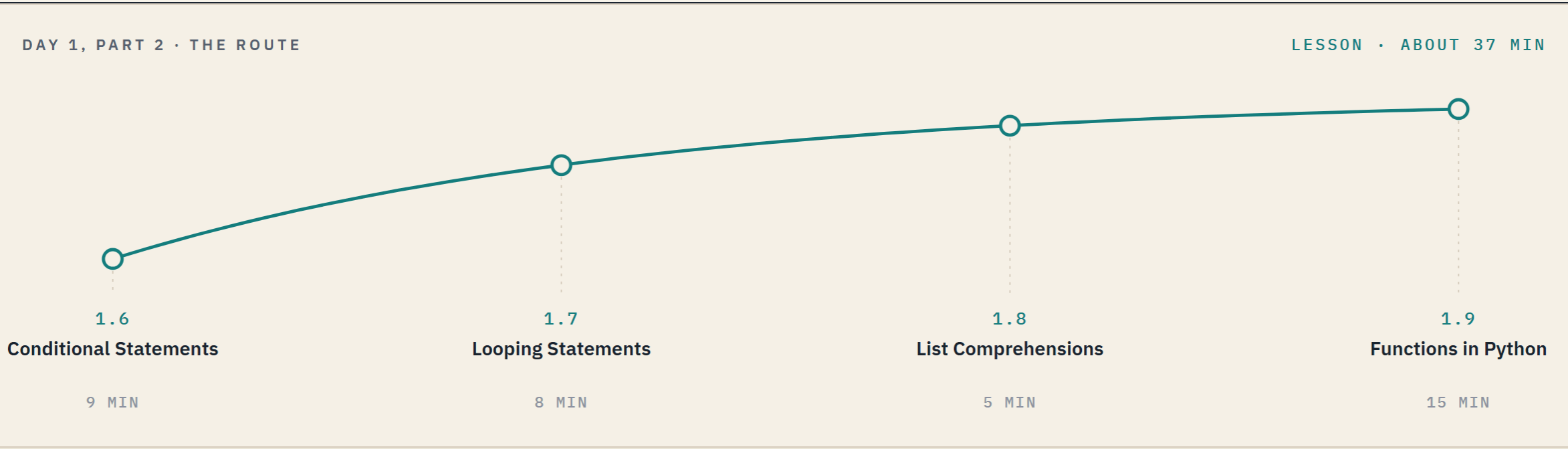


FIGURE 1 Four sections, in order, with about how long each takes.

Time. About 37 minutes for the lesson, then about 15 for the five exercises.

1.6 Conditional Statements

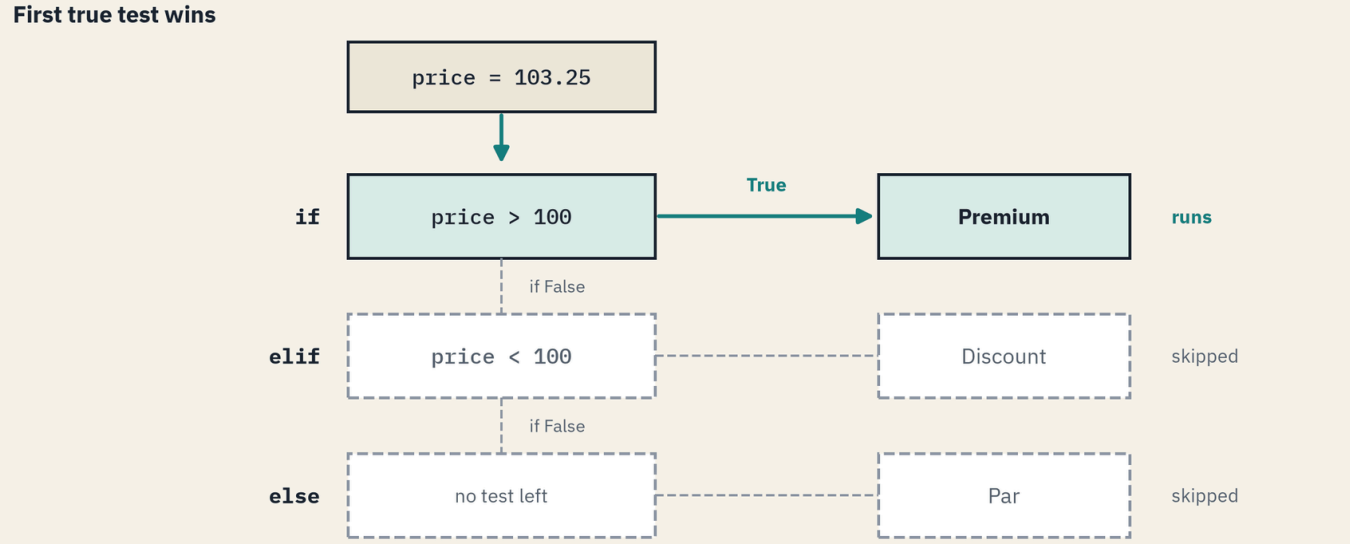


FIGURE 2 DNMR at 103.25: price > 100 is True, so Premium runs; the elif and the else are skipped.

PREMIUM, DISCOUNT, OR PAR

```
price = 103.25
if price > 100:
    print('DNMR trades at a premium')
elif price < 100:
    print('DNMR trades at a discount')
else:
    print('DNMR trades at par')
```

```
DNMR trades at a premium
```

EXCEL SIDE BY SIDE

Excel side by side. With the price in A1, a nested IF: `=IF(A1>100,"Premium",IF(A1<100,"Discount","Par"))`.

<= is one comparison. Python has more.

Compare and Combine

COMPARE AND COMBINE

PYTHON	ASKS	EXCEL
`==`	equal?	`=`
`!=`	not equal?	`<>`
`<` and `<=`	less? less or equal?	`<` and `<=`
`>` and `>=`	greater? greater or equal?	`>` and `>=`
`in` and `not in`	one of the items?	`=COUNTIF(A1:A4,"DNMR")>0`
`and`	both sides true?	`=AND(A1>10,B1>100)`
`or`	at least one side true?	`=OR(A1<=10,B1>100)`
`not`	flips True and False	`=NOT(A1<=10)`

COMPARE

```
score = rating_score['BB+']
print(score == 11, score != 11, score <= 10)
print('DNMR' in ticker_list, 'ZZZZ' not in ticker_list)
```

```
True False False
True True
```

COMBINE

```
if score == 10 or score == 11:
    print('DNMR sits on the investment grade line')
else:
    print('DNMR is clear of the line')
```

```
DNMR sits on the investment grade line
```

Repeat one sum over every bond.

1.7 Looping Statements

A LOOP OVER FIVE PRICES

```
face = 1500000
for test_price in range(96, 101):
    value = face * test_price / 100
    print(f'At {test_price} the position is worth ${value:,.2f}')
```

```
At 96 the position is worth $1,440,000.00
At 97 the position is worth $1,455,000.00
At 98 the position is worth $1,470,000.00
At 99 the position is worth $1,485,000.00
At 100 the position is worth $1,500,000.00
```

AN IF INSIDE THE LOOP

```
for ticker, rating in zip(ticker_list, rating_list):
    if rating_score[rating] <= 10:
        print(f'{ticker} ({rating}) is investment grade')
    else:
        print(f'{ticker} ({rating}) is high yield')
```

```
QVNE (BBB) is investment grade
DNMR (BB+) is high yield
PLWK (A-) is investment grade
HLVF (B+) is high yield
```

A loop can also walk a list, or a dictionary.

Zip, Then Total

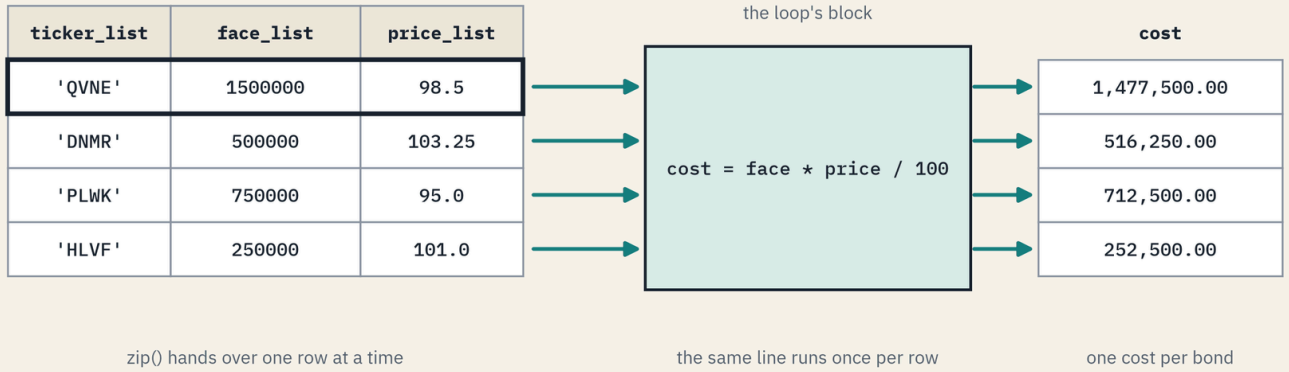


FIGURE 4 zip() hands each bond's ticker, face, and price to one line, and four costs come out.

STEP THROUGH COLUMNS TOGETHER

```
for ticker, face, price in zip(ticker_list, face_list, price_list):
    cost = face * price / 100
    print(f'{ticker} cost ${cost:,.2f}')
```

```
QVNE cost $1,477,500.00
DNMR cost $516,250.00
```

A RUNNING TOTAL

```
total_cost = 0
for face, price in zip(face_list, price_list):
    total_cost += face * price / 100
print(f'The book cost ${total_cost:,.2f}')
```

```
The book cost $2,958,750.00
```

EXCEL SIDE BY SIDE

Excel side by side. Faces in B2:B5 and prices in C2:C5: `=SUMPRODUCT(B2:B5,C2:C5)/100` gives it in one cell.

1.8 List Comprehensions

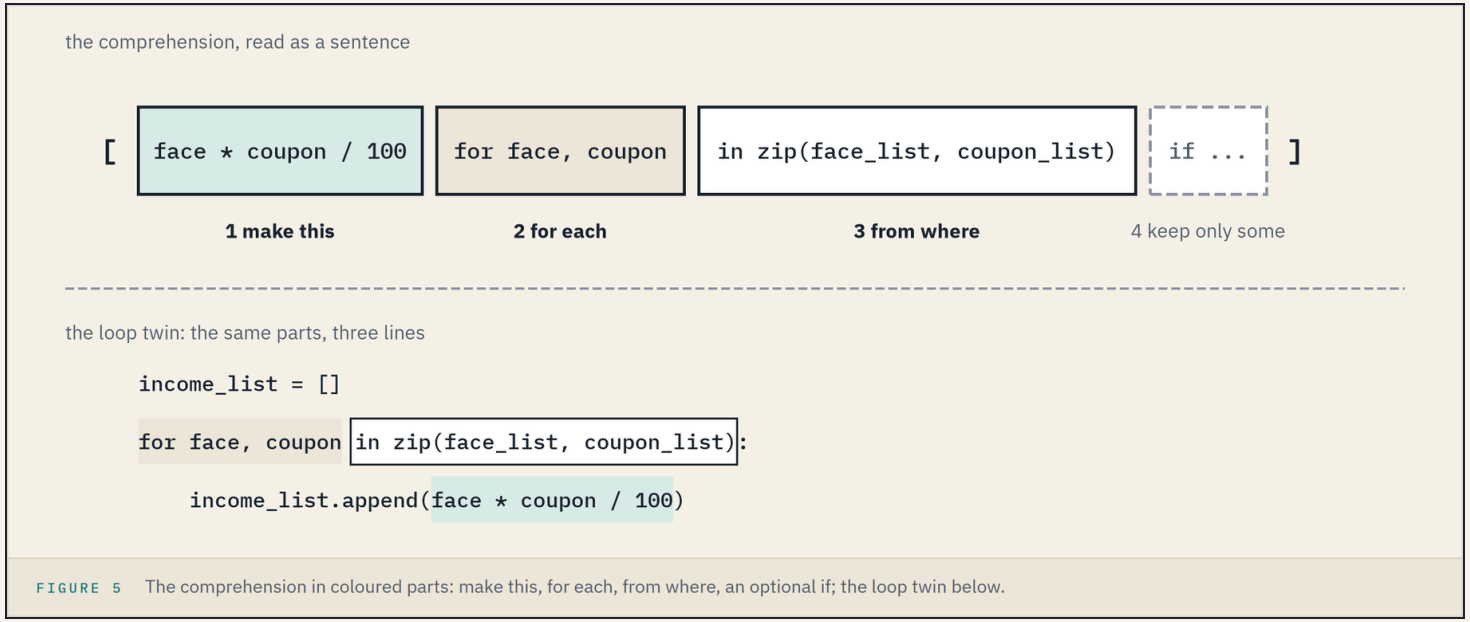


FIGURE 5 The comprehension in coloured parts: make this, for each, from where, an optional if; the loop twin below.

ONE LINE

```
income_list = [face * coupon / 100 for face, coupon in zip(face_list, coupon_list)]
print(income_list)
```

```
[78750.0, 30625.0, 35625.0, 18750.0]
```

THE LOOP IT REPLACES

```
income_list = []
for face, coupon in zip(face_list, coupon_list):
    income_list.append(face * coupon / 100)
print(income_list)
```

```
[78750.0, 30625.0, 35625.0, 18750.0]
```

EXCEL SIDE BY SIDE

Excel side by side. Tickers in A2:A5 and scores in F2:F5: `=FILTER(A2:A5,F2:F5<=10)` keeps the same two; `FILTER` needs Excel 2021 or later, or Microsoft 365.

1.9 Functions in Python

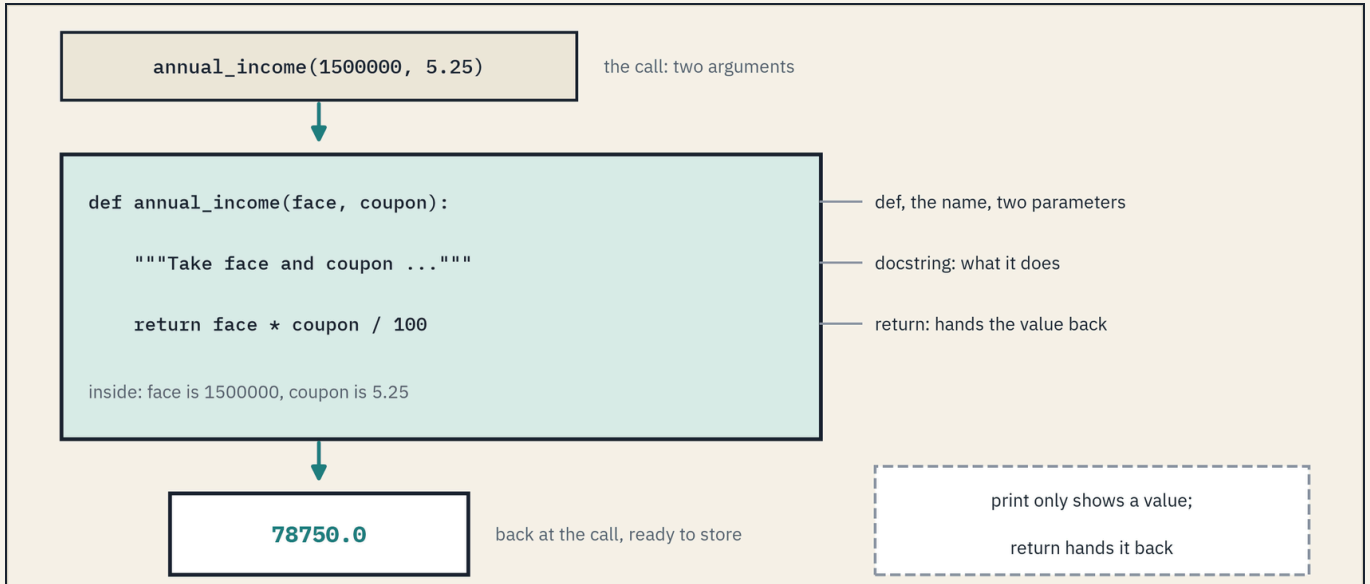


FIGURE 6 annual_income as a machine: the call goes in, def, docstring and return labelled, 78750.0 comes out.

CALL IT, STORE WHAT IT HANDS BACK

```
qvne_income = annual_income(1500000, 5.25)
print(f'QVNE pays ${qvne_income:,.2f} a year')
```

```
QVNE pays $78,750.00 a year
```

PRINT IS NOT RETURN

```
result = describe_bond('DNMR', 6.125, 2034, 103.25)
print(result)
```

```
DNMR 6.125% of 2034 is priced at 103.25
None
```

EXCEL SIDE BY SIDE

Excel side by side. In Microsoft 365, `LAMBDA` does this job: in the Name Manager, name `=LAMBDA(face,coupon,face*coupon/100)` as `ANNUAL_INCOME`, and `=ANNUAL_INCOME(1500000,5.25)` gives 78750.

You Can Now

- Label a bond: premium or discount, investment grade or high yield.
- Repeat a calculation over every bond, and keep a running total.
- Build a list in one line, choosing or filtering as it goes, and draw it.
- Write a function with parameters, a default, and a **`return`**.
- Sort a list, and read **`lambda`**, **`*args`**, and **`**kwargs`**.

THE NOTE ASKED	ANSWER
Which bonds are investment grade	QVNE and PLWK; DNMR and HLVF are high yield
Cost every position in one go	one loop: 2,958,750.00 dollars
Make the coupon sum reusable	**`annual_income(face, coupon)`**: 78,750.00 dollars a year for QVNE

NEXT
WEEK 1 · DAY 2

Files and Folders

Using Python to find, create, and organize the files a bond desk works with.